

Research and Development of an Automated System for Concept Drift Detection and Adaptive Machine Learning Model Updating

Master's Thesis

Student: Le Phuc Duc

Supervisor 1: Assoc.Prof.Dr. Thoai Nam

Supervisor 2: Dr. Nguyen Quang Hung

Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

2026



Presentation Outline

- 1 Introduction
- 2 Background: The MMD Drift Signal
- 3 Layer 1: Detection
- 4 Layer 2: Classification
- 5 Layer 3: Adaptation
- 6 Evaluation & Results
- 7 System Deployment
- 8 Conclusion



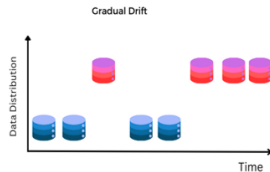
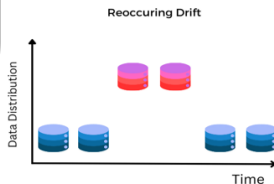
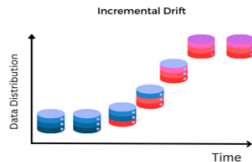
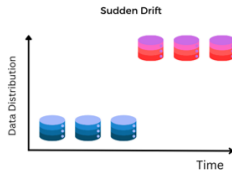
The Core Problem: Concept Drift

Mathematical Definition

$$P_{\text{train}}(X, Y) \neq P_{\text{online},t}(X, Y)$$

Real-world Impact:

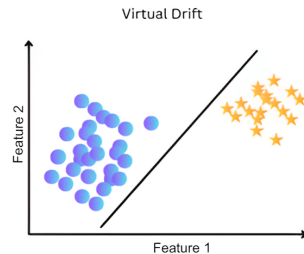
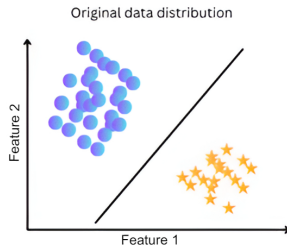
- Banking Fraud (False Alarms $\uparrow$)
- Predictive Maintenance (Accuracy $\downarrow$)



Concept Drift: Distribution vs Pattern

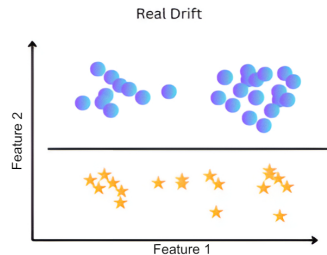
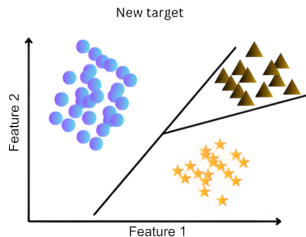
Virtual Drift:

Only $P(X)$ changes.

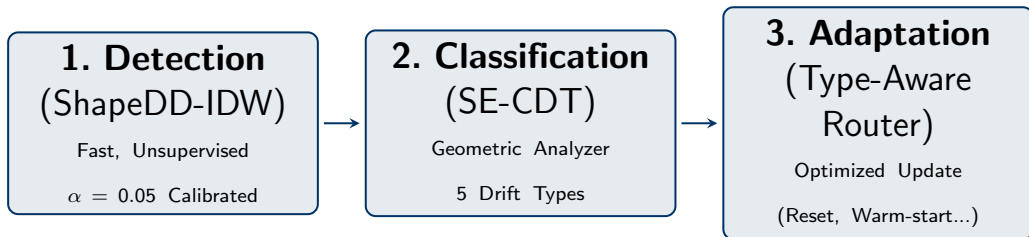


Real Drift:

Decision boundary $P(Y|X)$ changes.



A Three-Stage Pipeline

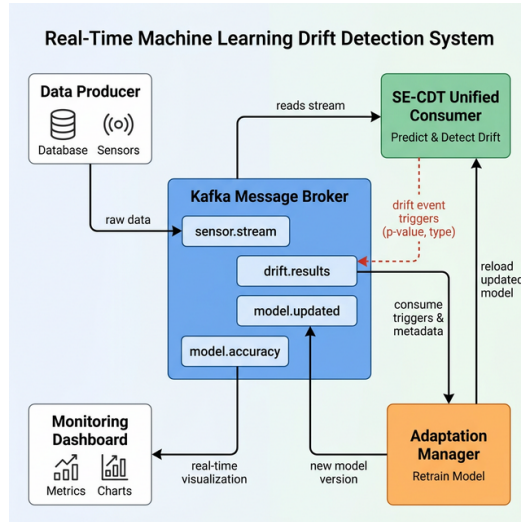


- **Goal 1:** Fast & reliable detection (near-zero false alarms).
- **Goal 2:** Unsupervised drift type identification (no labels needed).
- **Goal 3:** End-to-end continuous evaluation on Apache Kafka.

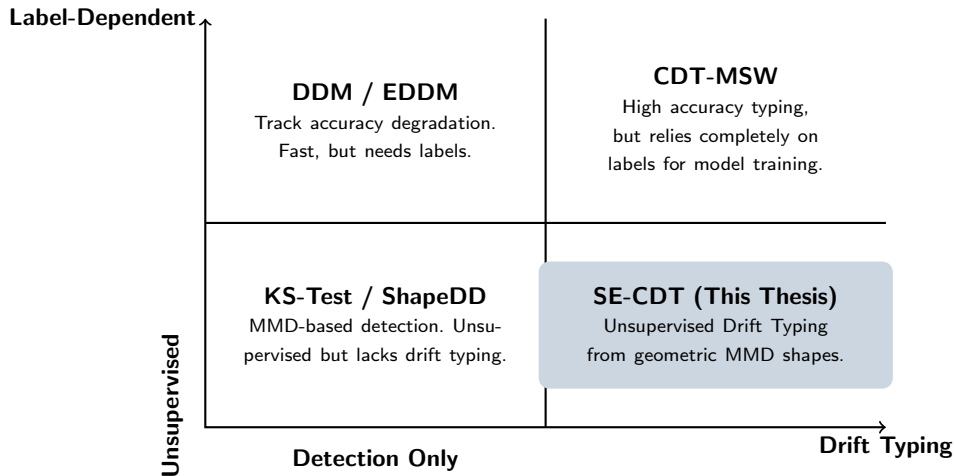
SE-CDT System Architecture

3-Layer Design:

- **L1: Detect**
(When?)
- **L2: Classify**
(What?)
- **L3: Adapt**
(How?)



Related Works: The Gap in Current Literature



Contributions of This Thesis

- ① **Detection (Layer 1):** A calibrated, label-free detector — ShapeDD with IDW optimal weighting and a **Gamma-null** p-value that replaces the permutation test for a $\sim 7\times$ speedup.
- ② **Classification (Layer 2): SE-CDT** — a fully *unsupervised* drift-type classifier that reads 9 geometric/temporal shape attributes (plus a Variance Ratio) of the MMD trace instead of accuracy labels.
- ③ **Adaptation (Layer 3):** A **type-aware** routing policy mapping each drift type to its cheapest correct update, validated end-to-end on an Apache Kafka prototype.



Why Unsupervised Detection?

The problem with supervised detection:

- Requires ground-truth labels immediately after predictions.
- In many real-world IIoT / stream scenarios, labels are delayed or expensive.

The Unsupervised Alternative:

- Monitor the **raw data distribution** $P(X)$ natively without labels.
- We map all data points into a high-dimensional function space (RKHS).
- Then, we compare their “centers of mass” to calculate a **Drift Signal**.



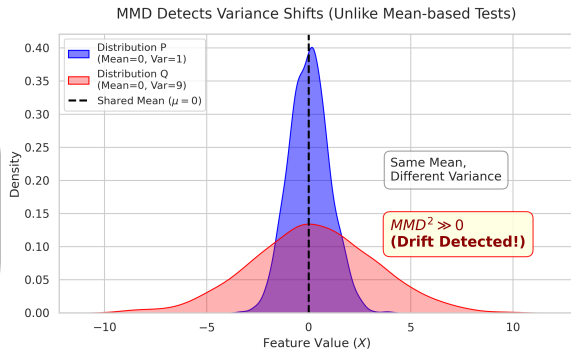
Maximum Mean Discrepancy (MMD)

Compare mean embeddings in an RKHS

MMD Statistic

$$\widehat{\text{MMD}}^2 = \underbrace{\text{Within } P}_{\text{self-sim}} + \underbrace{\text{Within } Q}_{\text{self-sim}} - \underbrace{\text{Cross } P \leftrightarrow Q}_{\text{dis-sim}}$$

- $\widehat{\text{MMD}}^2 \approx 0$: Distributions are same
- $\widehat{\text{MMD}}^2 \gg 0$: **Drift Candidate**



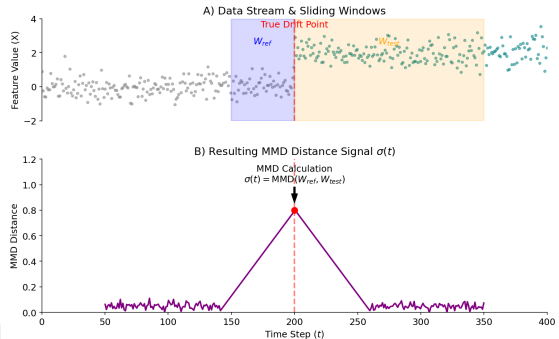
Online MMD Estimation via Sliding Window

Stream-to-Trace Mapping:

- A sliding window of size $2l_1$ moves over the raw data stream.
- Split into reference window W_{ref} (first half) and test window W_{test} (second half).
- MMD is computed between W_{ref} and W_{test} at each step.
- The result is a continuous MMD trace $\sigma(t)$ tracking distributional changes over time.

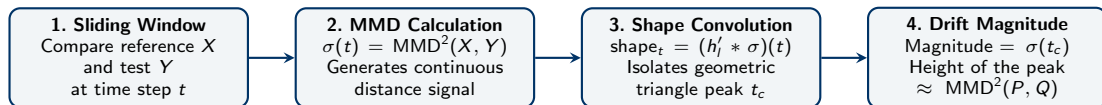
Detection Signal

When the window covers stable data, $\sigma(t)$ fluctuates around a small noise floor. When crossing a drift, $\sigma(t)$ rises and forms a peak.



Concept: Drift Magnitude & Shape Detection

High-Level Workflow: From Sliding Windows to Drift Magnitude



- **Continuous Tracking:** MMD acts as an unsupervised distance metric computed continuously over time.
- **Peak Isolation:** ShapeDD convolves the trace with a derivative-of-triangle filter h'_l to find where the slope flips.
- **Drift Magnitude:** The peak height $\sigma(t_c)$ reflects the exact intensity of the distribution shift.

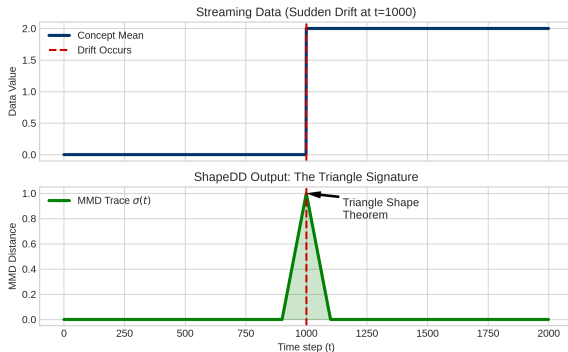
MMD vs ShapeDD: The Relationship

MMD = Distance Metric

- Calculates distance between two windows at a single moment.
- Peak height represents the **Drift Magnitude** ($\approx \text{MMD}^2(P, Q)$).

ShapeDD = Detection System

- Computes MMD continuously to generate a trace $\sigma(t)$.
- **Innovation:** Looks for a **geometric triangle shape**.
- **Why?** Linear contamination as the sliding window steps over the drift point.



ShapeDD — Shape-Based Drift Detection

Triangle Shape Theorem:

- A sudden drift reliably produces a triangular MMD trace $\sigma(t)$.

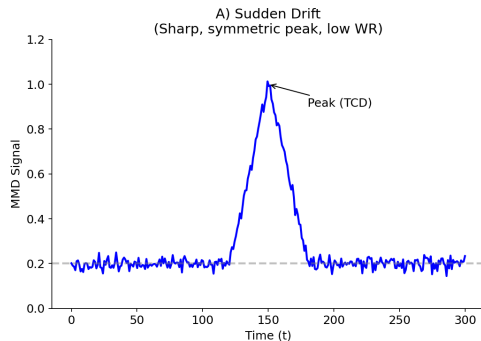
Processing pipeline:

- 1 Maintain a sliding window.
- 2 Build the kernel matrix.
- 3 Compute MMD between the two halves.
- 4 Test significance with a permutation test.

Bottleneck

The permutation test re-builds the kernel thousands of times
→ heavy compute overhead.

Solution: Combine **IDW optimal weighting** (for sensitivity) and **Gamma-Null Approximation** (alternative to permutation).



Inverse Density-Weighted MMD (IDW-MMD)

Limitation of standard MMD:

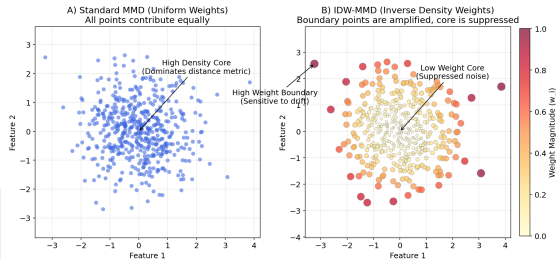
- Dense majority clusters dominate.
- Early drifts at distribution boundaries are washed out.

Solution: IDW Weighting

Down-weight dense core data; amplify sparse boundary data.

The Result

High sensitivity to early boundary changes without increasing false alarms on the bulk data.



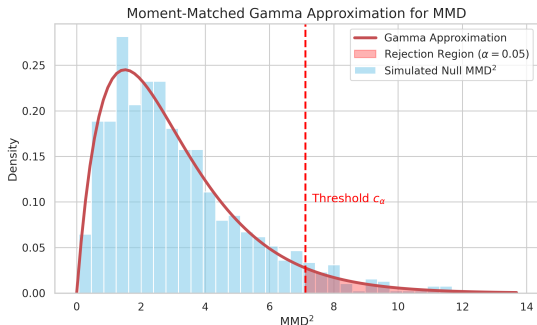
Statistical Calibration (Gamma-Null Approximation)

The Permutation Bottleneck:

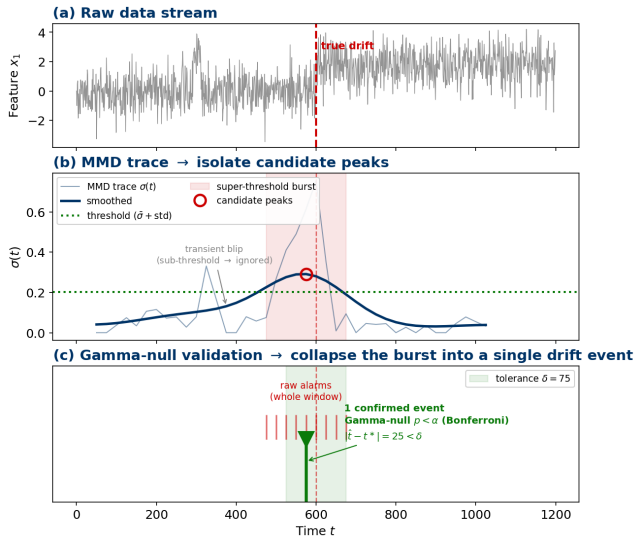
- Standard ShapeDD runs $B = 2500$ permutations to estimate p-value.
- Each permutation re-builds the kernel matrix $\rightarrow$ extremely slow.

Gamma-Null Approximation Solution:

- Under H_0 , the MMD statistic is approximately Gamma-distributed.
- We estimate the first two moments (mean & variance) from a small number of bootstrap shuffles ($B = 20$), reusing the kernel matrix.
- Matches accuracy while achieving a **7–10 $\times$** end-to-end speedup.



ShapeDD-IDW in Action: MMD Trace $\rightarrow$ Single Drift Event



The detector, end-to-end:

- 1 (a) Stream $\rightarrow$ compute the IDW-MMD trace $\sigma(t)$.
- 2 (b) Isolate peaks: smooth $\sigma(t)$, keep points above $\bar{\sigma} + \text{std}$. A short **blip** stays sub-threshold $\rightarrow$ ignored.
- 3 (c) **Validate & group**: the Gamma-null p -value (Bonferroni-adjusted) confirms the peak; the whole super-threshold **burst collapses to one event** within tolerance δ .

Outcome

One calibrated, label-free detection per true drift — no permutation test, few false alarms.

CDT-MSW — Supervised Classification Baseline

CDT-MSW (Guo et al., 2022) groups drift into:

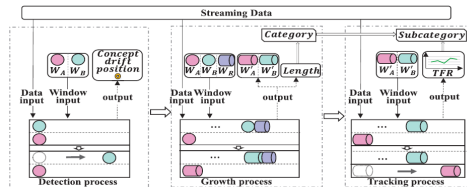
- **TCD (Transient)**: Sudden, Blip, Recurrent.
- **PCD (Progressive)**: Gradual, Incremental.

Growth Process:

- Tracks the variance of accuracy degradation over a multi-sliding window.
- Short degradation → TCD; sustained degradation → PCD.

Limitation

CDT-MSW needs **ground-truth labels** at every step to compute the accuracy curve — impractical for IIoT streams where labels arrive late or never.



Motivation for SE-CDT

Replace the accuracy curve with the *shape* of the MMD signal $\sigma(t)$. The same TCD/PCD distinction can then be made **without any labels at runtime**.

SE-CDT: 9 Statistical Shape Attributes

How can we type drift without ground-truth labels?

Read the *shape* of the MMD signal $\sigma(t)$ around the detection point. SE-CDT extracts exactly **9 statistical attributes**:

Geometric Attributes (Peak Structure)

- n_p (Peak Count)
- WR (Width Ratio)
- CV (Coefficient of Variation)
- SNR (Signal-to-Noise Ratio)
- PPR (Peak Proximity Ratio)
- DPAR (Dual-Peak Amplitude Ratio)

Temporal Attributes (Time Behaviour)

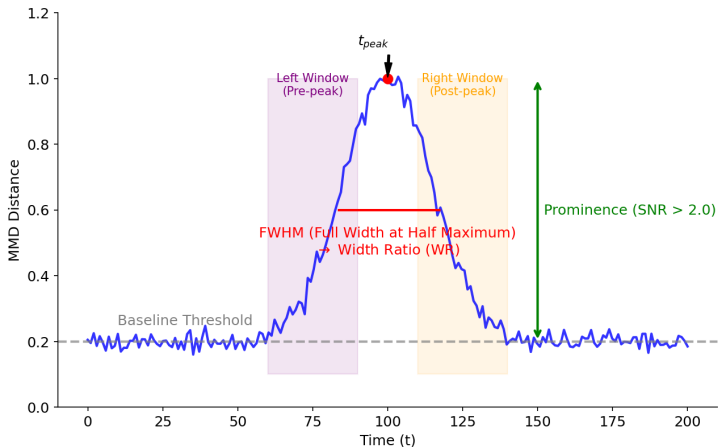
- LTS (Linear Trend Strength)
- SDS (Step Detection Score)
- MS (Monotonicity Score)

+ **one raw-data signal** — **Variance Ratio (VR)**: computed from the *raw* window X , *not* the trace shape $\sigma(t)$ — the dedicated slow-drift tie-breaker (next slide).



SE-CDT: Peak Anatomy & Feature Extraction

Visualizing Peak Geometry and Baseline Noise:



SE-CDT: Variance Ratio (VR) for Slow Drifts

The Challenge: Gradual and Incremental drifts both produce wide, flat MMD humps — shape features alone cannot separate them.

Key idea — read the variance of the raw window:

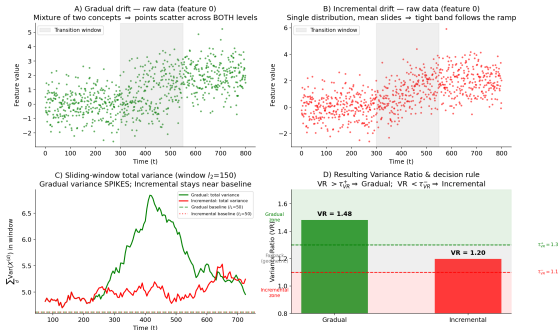
- **Gradual** = mixture of two concepts $\Rightarrow$ **variance spike**.
- **Incremental** = one distribution shifting $\Rightarrow$ **flat variance**.

(Law of Total Variance: the between-concept term is the drift mixer.)

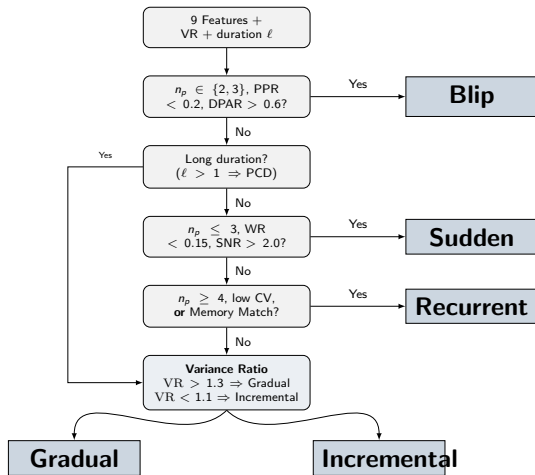
Variance Ratio (VR)

$$VR = \frac{\max_i \text{Var}(X_{[i, i+l_2)})}{\text{Var}(X_{[0, l_1)})}$$

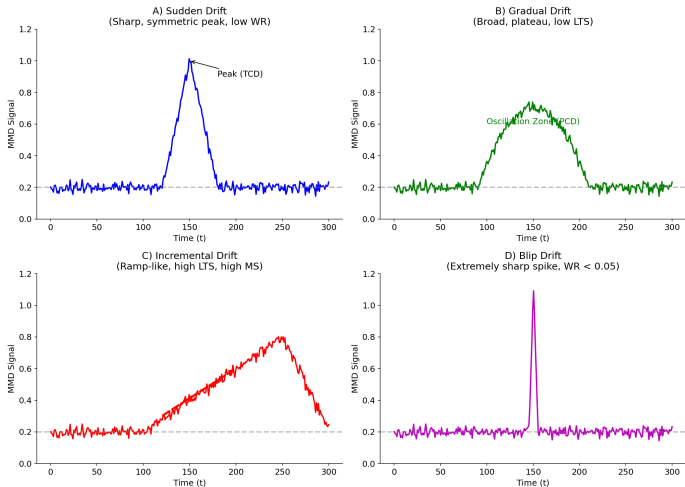
$VR > 1.3 \Rightarrow$ **Gradual**; $VR < 1.1 \Rightarrow$ **Incremental**;
otherwise $\Rightarrow$ fall back to geometry.



Rule-based Logic to Identify Drift Types



SE-CDT Drift Signatures: Visualised



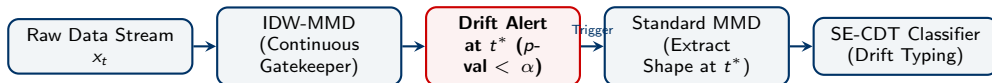
Hybrid Pipeline: IDW-MMD to Detect, Standard MMD to Classify

Why use two MMD variants instead of one?

- **IDW-MMD:** Very sharp, filters out noise and stabilizes Type-I error ($\alpha = 0.05$) on stable windows. But its density weights flatten gradual transitions.
- **Standard MMD:** Slower but preserves natural geometry of MMD trace (rising trends, duration info) needed for classification.

Gated Execution Strategy

- **L1 Detect:** Run cheap **IDW-MMD** continuously.
- **L2 Classify:** Compute **Standard MMD** only at detection point t^* on the candidate window to extract shape.



Optimizing Update Cost

Detected Drift Type	Adaptation Policy
Sudden	Full Reset
Incremental	Continuous Update
Gradual	Weighted Window
Recurrent	Model Reuse (from Memory)
Blip	Ignore (Do Nothing)

Goal

Minimize catastrophic forgetting and compute overhead by avoiding a "one-size-fits-all" retraining policy.

Kafka Integration

Triggers downstream microservices to hot-swap or fine-tune models dynamically.



14 Evaluation Datasets (Grouped):

- **Sudden** $P(X)$: Gaussian Shift (Moderate), GCS (3 concepts), $4 \times$ Random Uniform (Sensitivity tests: Mild to Ultra Severe).
- **Blip** $P(X)$: RBF Blips (short-term recurring centroid shifts).
- **Real Drift** $P(Y|X)$ (**Control Group**): Standard SEA, Rotating Hyperplane, LED Abrupt (no feature drift; to verify negative controls).
- **Semi-Synthetic**: Electricity Semi-Synthetic (real features, 10 injected drift points).
- **Slow & Cycle**: Gaussian Gradual (linear transition), GRS (Recurrent $A \leftrightarrow B$), Gaussian Stationary (H_0 calibration).

Experimental Protocol

- **Runs**: 30 independent seeds per stream to ensure statistical stability.
- **Layer 1 (Detection)**: Tolerance $\delta = 75$, cooldown = 150. Measured via F1-score, EDR, and False Positives.
- **Layer 2 (Classification)**: Oracle Mode (1800 events total: 6 scenarios $\times$ 30 seeds $\times$ 10 events) to measure CAT and SUB accuracy.
- **Layer 3 (Adaptation)**: Prequential accuracy (test-then-train) on Stepping and Sudden streams.

Classification Setup (6 scenarios $\times$ 30 seeds, oracle mode)

Streams (5-D Gaussian, 1800 events total):

Scenario	Composition
Mixed A	Sudden $\rightarrow$ Gradual $\rightarrow$ Recurrent
Mixed B	Blip $\rightarrow$ Incremental $\rightarrow$ Sudden
Repeated Sudden	10 $\times$ Sudden
Repeated Gradual	10 $\times$ Gradual
Repeated Incremental	10 $\times$ Incremental
Repeated Recurrent	10 $\times$ Recurrent

6 scenarios $\times$ 30 seeds $\times$ 10 events = **1800 events**. Each non-recurrent event shifts a different feature subset, so new concepts are genuinely distinct (Lu et al. definition).

Oracle mode

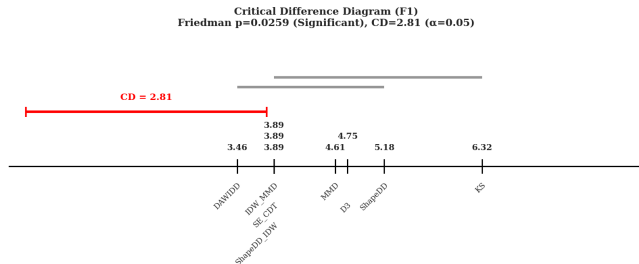
The classifier receives the ground-truth drift *position* and is scored only on the *typing* step. This isolates classification skill from detection skill — the same way the detection protocol ($\delta = 75$, cooldown) isolates detection.

Two metrics

CAT — TCD vs PCD (the decision that drives adaptation).
SUB — exact 5-class subtype (micro / macro).



Result 1a: Drift Detection Performance



Statistical Significance (Nemenyi Test):

- **Meaning:** Post-hoc pairwise comparison of detectors across all 14 datasets.
- **Critical Difference (CD):** Threshold of 2.81 ($\alpha = 0.05$).
- **Interpretation:** Groups connected by a thick line (like DAWIDD, MMD, and SE-CDT) have **no statistically significant difference** in performance.
- **Result:** SE-CDT (rank 3.89) is statistically tied with state-of-the-art DAWIDD (rank 3.46).

Key Improvement

Note: SE-CDT matches SOTA F1 detection performance, but achieves a lower false positive rate.

Result 1b: Comprehensive Detection Performance

Overall Metrics across all evaluation streams:

Method	Precision	Recall (EDR)	F1-Score	Delay	False Pos.
DAWIDD	0.435	0.734	0.532	38	10.3
IDW_MMD	0.432	0.737	0.531	36	9.6
SE_CDT	0.432	0.737	0.531	36	9.6
ShapeDD_IDW	0.432	0.737	0.531	36	9.6
MMD	0.429	0.734	0.527	35	10.5
ShapeDD	0.377	0.749	0.491	34	13.2
D3	0.558	0.472	0.489	16	0.6
KS	0.224	0.775	0.335	34	27.2

- **Synthetic $P(X)$ drifts** (Gaussian, Random Uniform): strong F1 on moderate-to-severe shifts (0.69–0.74), degrading gracefully on the mildest case (Random Mild, 0.51), with $FP \leq 10$.
- **Recurrent & blip streams** (Rbfblips, Stagger Recurrent): reliably detected ($F1 \approx 0.74$ –0.76) with minimal false alarms.
- **Real-drift controls** (SEA, Hyperplane, LED): low F1 *by design* — no $P(X)$ shift is present, confirming the detector does not false-fire on pure $P(Y|X)$ drift.



Result 1b: Per-Dataset Detection F1 (1/2)

Part 1: Basic & Gradual/Mild Drift

Method	Electricity	Gaussian Gradual Synthetic	Gaussian Moderate	Random Mild
DAWIDD	0.250	0.472	0.753	0.594
IDW_MMD	0.383	0.470	0.736	0.511
SE_CDT	0.383	0.470	0.736	0.511
ShapeDD_IDW	0.383	0.470	0.736	0.511
MMD	0.291	0.462	0.740	0.566
ShapeDD	0.267	0.451	0.700	0.529
D3	0.213	0.029	0.998	0.012
KS	0.263	0.315	0.356	0.335

Part 2: Random Uniform & Hyperplane

Method	Random Moderate	Random Severe	Random Ultra Severe	Hyperplane
DAWIDD	0.728	0.739	0.738	0.136
IDW_MMD	0.688	0.708	0.712	0.179
SE_CDT	0.688	0.708	0.712	0.179
ShapeDD_IDW	0.688	0.708	0.712	0.179
MMD	0.685	0.707	0.709	0.169
ShapeDD	0.619	0.636	0.638	0.197
D3	0.244	0.931	0.933	0.000
KS	0.407	0.434	0.434	0.240

Result 1b: Per-Dataset Detection F1 (2/2)

Part 3: Led, Rbfbliips & Stagger

Method	Led Abrupt	Rbfbliips	Stagger	Stagger Recurrent Explicit
DAWIDD	0.130	0.749	0.750	0.749
IDW_MMD	0.148	0.744	0.755	0.753
SE_CDT	0.148	0.744	0.755	0.753
ShapeDD_IDW	0.148	0.744	0.755	0.753
MMD	0.127	0.747	0.742	0.739
ShapeDD	0.163	0.670	0.681	0.679
D3	0.000	1.000	0.998	1.000
KS	0.071	0.371	0.459	0.458

Part 4: Sea & Overall Mean F1

Method	Standard Sea	Mean
DAWIDD	0.129	0.532
IDW_MMD	0.118	0.531
SE_CDT	0.118	0.531
ShapeDD_IDW	0.118	0.531
MMD	0.163	0.527
ShapeDD	0.151	0.491
D3	0.000	0.489
KS	0.206	0.335

Result 1c: Type-I Error on Stationary Data

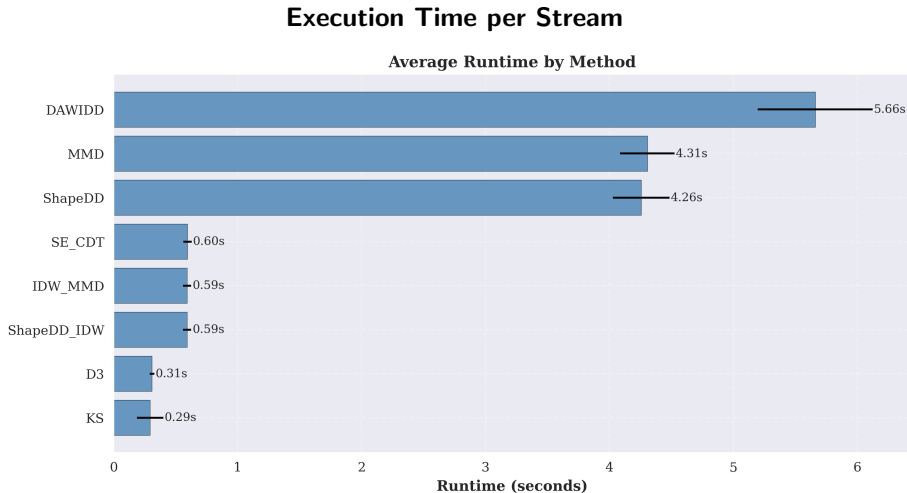
Type-I Error at nominal $\alpha = 0.05$

Distribution	Standard MMD (perm)	IDW-MMD (Gamma null)	SE-CDT (composite)
Gauss-iid d=5	0.025 $\pm$ 0.022	0.040 $\pm$ 0.027	0.115 $\pm$ 0.044
Gauss-iid d=10	0.055 $\pm$ 0.032	0.050 $\pm$ 0.030	0.080 $\pm$ 0.038
Gauss-AR1 d=5	0.045 $\pm$ 0.029	0.075 $\pm$ 0.037	0.140 $\pm$ 0.048

Evaluation on Stationary Streams:

- Evaluated across 3 stationary datasets (no drift) over 30 runs to measure false alarms.
- **IDW-MMD Calibration:** Holds extremely close to the nominal rate ($\alpha = 0.05$), ranging between 0.025–0.075.
- **Composite Trace (SE-CDT):** Becomes anti-conservative (0.080–0.140) due to peak selection and sliding-window dependency.

Result 1d: Runtime Efficiency (1/2)



Result 1d: Runtime Efficiency (2/2)

Execution Time per Stream

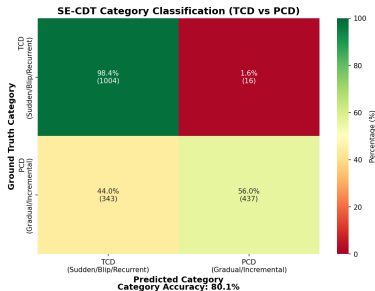
Method	Runtime (s/stream)	Throughput (samples/s)	Speedup vs ShapeDD
KS	0.29	33,990	14.47×
D3	0.31	32,300	13.75×
ShapeDD-IDW	0.59	16,858	7.18×
IDW-MMD (standalone)	0.59	16,849	7.17×
SE-CDT	0.60	16,764	7.14×
ShapeDD (original)	4.26	2,349	1.00×
MMD	4.31	2,322	0.99×
DAWIDD	5.66	1,767	0.75×

Computational Speedup

Gamma-null approximation makes SE-CDT roughly **7×** **faster** than permutation-based ShapeDD (**0.60 s** vs. **4.26 s** per 10k samples) and standard MMD (4.31 s), bringing kernel-based detection into the latency class of lightweight tests like KS and D3.



Result 2: Unsupervised Classification (SE-CDT in Oracle Mode)



Headline — label-free typing

CAT = 80.1% (TCD vs PCD — the decision that drives adaptation), at **92.6% recall**, with **no labels**.

Per-type subtype accuracy (1800 events):

Drift Type	Acc.	Notes
Sudden	82.4%	Sharp, well-separated peaks
Recurrent	71.5%	Enabled by Concept Memory
Blip	60.8%	Multi-peak tolerance rule
Incremental	30.3%	Overlaps Gradual on VR
Gradual	27.2%	57% absorbed into Recurrent

Two enhancements added

Concept Memory: ring buffer of post-drift snapshots, matched via Standard MMD. Recovers Recurrent classification when the same concept reappears.

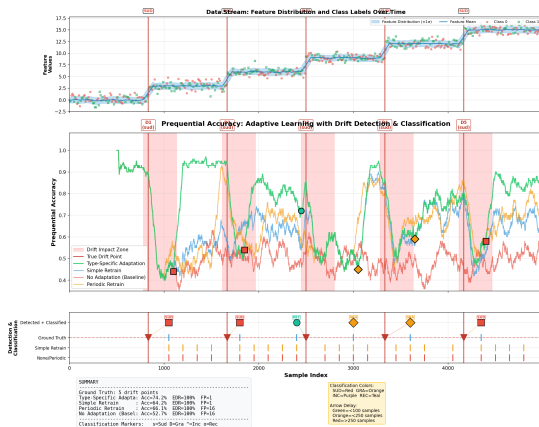
Online Self-Calibration: rolling quantile baseline that adapts SNR/WR/CV thresholds when no drift is present, improving robustness on noisy streams.

Honest limitation

Subtype (SUB) is 55.3% — the two slow types (Gradual, Incremental) overlap on the Variance Ratio and are the weak point. Supervised CDT-MSW reaches higher SUB but needs labels (head-to-head in backup).



Result 3: Model Adaptation (Normal / Streaming Mode)

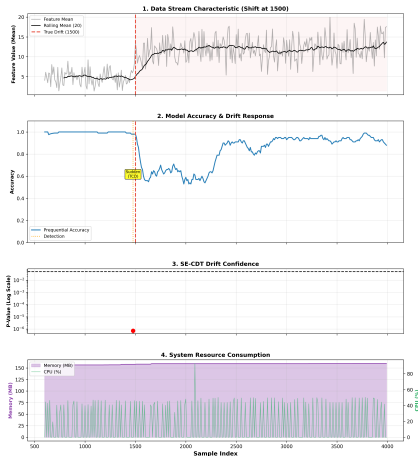


Strategy	Step.	Sudd.
Type-Specific	71.4%	70.8%
Periodic Retrain	64.0%	63.7%
Simple Retrain	63.0%	62.2%
No Adaptation	54.8%	53.7%

Why Type-Specific helps

- **Avoids Waste:** Resetting on gradual drift discards history; periodic retrains ignore signals.
- **Optimal Update:** Sudden → full reset, Gradual → weighted-window (warm-start).
- **Gains: +16.6 pp (Stepping) and +17.1 pp (Sudden) vs No Adaptation ($p < 10^{-15}$).**

End-to-End Deployment Prototype on Apache Kafka



Summary

- ➊ **Detection:** ShapeDD-IDW with Gamma-null p-value provides a calibrated, label-free test with an end-to-end speedup.
- ➋ **Classification:** SE-CDT operates label-free, achieving high Category Accuracy (80.1%) and event recall (92.6%).
- ➌ **Adaptation:** Type-Specific routing yields substantial gains (+17.1 pp on Sudden) over no adaptation.
- ➍ **System:** Validated through an end-to-end Kafka prototype deployment.

Key Limitations

- **Subtype Confusion:** Gradual and Incremental shifts remain challenging (27.2% / 30.3% accuracy) due to overlap on the Variance Ratio.
- **Empirical Thresholds:** The Concept Memory threshold ($\mu = 0.15$) was chosen empirically based on the H0 noise floor.
- **Adaptation Evaluation Scope:** Tested heavily on sudden-dominated streams; robustness on purely gradual/incremental scenarios requires further study.

**THANK YOU
FOR LISTENING!**

Q & A

Student: Le Phuc Duc

Supervisor 1: Assoc.Prof.Dr. Thoai Nam

Supervisor 2: Dr. Nguyen Quang Hung

Backup: SE-CDT Classification Algorithm

Algorithm: SE-CDT — Unsupervised Drift Type Classification

Require: Drift magnitude signal $\sigma(t)$ from Detection Module

Require: Raw data window W for Variance Ratio

Ensure: Drift type: {TCD-Blip, TCD-Sudden, TCD-Recurrent, PCD-Gradual, PCD-Incremental}

```
1:  $\sigma_s \leftarrow \text{GaussianFilter}(\sigma, \sigma_g)$ 
2:  $P \leftarrow \text{FindPeaks}(\sigma_s, \bar{\sigma}_s + \alpha \sigma_{std})$ 
3: Compute features:  $n_p \leftarrow |P|$ , WR, CV, SNR, PPR, DPAR, LTS, SDS, MS
4:  $VR \leftarrow \text{VarianceRatio}(W)$ 
5:  $\ell \leftarrow \text{EstimateDriftDuration}(\sigma_s)$ 
6: if  $n_p \leq \tau_n^{\text{blip}}$  and  $WR < \tau_{WR}^{\text{blip}}$  and  $\text{BlipProfile}(\dots)$  then
7:   return TCD-Blip
8: end if
9: if  $\ell > \tau_\ell$  then
10:   if  $\text{IsIncremental}(VR, LTS, \dots)$  then
11:     return PCD-Incremental
12:   else
13:     return PCD-Gradual
14:   end if
15: end if
16: if  $n_p \leq \tau_n^{\text{sud}}$  and  $WR < \tau_{WR}$  and  $\text{SNR} > \tau_{\text{SNR}}$  then
17:   return TCD-Sudden
18: end if
19: if  $n_p \geq \tau_n^{\text{rec}}$  and  $CV < \tau_{CV}$  and  $LTS < \tau_{LTS}$  then
20:   return TCD-Recurrent
21: end if
22: if  $\text{IsIncremental}(VR, LTS, \dots)$  then
23:   return PCD-Incremental
24: else
25:   return PCD-Gradual
26: end if
```


Backup: IDW-MMD vs Optimally-Weighted MMD (Bharti et al.)

Question: “How is IDW-MMD different from the Optimally-Weighted MMD of Bharti et al. (2023)?”

Bharti et al. (OW-MMD)

- Domain: **likelihood-free inference** (ABC).
- Weights chosen by optimising against a target posterior.
- Requires solving a constrained convex optimisation per call.
- Not designed for streaming drift detection.

IDW-MMD (this work)

- Domain: **drift detection** in fast data streams.
- Weights from a closed-form heuristic:
 $w_i \propto 1/\sqrt{d_i + \epsilon}$.
- Cost: $O(n^2)$ for the kernel matrix, $O(n)$ extra for the weights — no optimisation step.
- Same *weighted MMD* family, different objective and very different runtime.

Summary: both are weighted MMD variants, but they target different problems (ABC vs streaming drift), use different weight functions, and have very different runtime profiles.



Backup: “F1 close to MMD baseline — where is the contribution?”

Question: “SE-CDT’s F1 (0.531) is only marginally above MMD (0.524). Where is the gain?”

F1 alone hides the trade-off:

	SE-CDT	MMD	KS
F1	0.531	0.524	0.335
FP	9.6	10.6	27.2
Recall	0.737	0.732	0.775
Precision	0.432	0.426	0.224

In a production setting:

- Frequent FPs → **alert fatigue**, lost trust in the monitor.
- Investigating one false alarm typically costs more than missing one event.
- A miscalibrated permutation-style test inflates either FPs or misses.

What SE-CDT actually adds:

- 1 **Single-test calibrated near $\alpha = 0.05$** — Standard MMD / IDW-MMD Type-I error 0.025–0.075; the SE-CDT composite is anti-conservative (0.080–0.140), reported honestly as a pipeline limitation.
- 2 **Classification module on top of detection** — gives a drift type, not just a flag.
- 3 **Modest runtime speedup over ShapeDD** thanks to the Gamma-null p-value (no kernel re-build per shuffle).
- 4 Stays **fully unsupervised** at runtime.



Backup: Why does Concept Memory matter for Recurrent drift?

Question: “Recurrent drift requires concept matching — how does Concept Memory provide it?”

The detection challenge:

- A Recurrent stream is a sequence of *repeated* concept switches; each switch produces an MMD peak that, in isolation, looks like a Sudden drift.
- Without memory of past concepts, repeated visits are scored independently $\rightarrow$ Recurrent collapses into the Sudden branch.

How Concept Memory resolves it:

- 1 During every *stable period*, take a snapshot of the current data window and store it in a ring buffer of size $K = 8$.
- 2 When a new drift fires, compute Standard MMD between the new post-drift window and every snapshot.
- 3 If the smallest $\text{MMD} < \mu = 0.15 \rightarrow$ override the

Why $\mu = 0.15$?

$\mu = 0.05$ falls inside the MMD noise floor (false matches between independent stationary windows). An empirical sweep on H0 calibration data shows $\mu = 0.15$ separates “revisited concept” from “new concept” reliably.

Remaining limitations

Two failure modes remain on the harder Recurrent variants:

- Recurrent events that are *too far apart* push the original snapshot out of the ring buffer.
- Repeated Gradual streams saturate after a few events, so the post-drift window matches

Backup: “CDT-MSW has higher CAT and SUB — is SE-CDT worse?”

Question: “CDT-MSW reports CAT 87.0% and SUB 86.7%, both higher than SE-CDT. Why frame SE-CDT as competitive?”

Side-by-side numbers

	SE-CDT	CDT-MSW
Labels	None	Required
CAT	80.1%	87.0%
SUB	55.3%	86.7%
Recall	92.6%	82.5%
Effective	74.2%	71.8%

Effective = CAT $\times$ Recall (probability of seeing a drift *and* typing it correctly). CDT-MSW Recall = $1 - \overline{\text{MDR}}$ from paper Table 5.

Reading the numbers in context:

- CDT-MSW reports CAT and SUB *conditionally* on drifts it detects; with paper recall $\approx 82.5\%$ it still misses $\sim 17\%$ of events.
- SE-CDT detects 92.6% of drifts and types every one of them without any labels.
- End-to-end, SE-CDT lands 2.4pp above CDT-MSW on effective accuracy (74.2% vs 71.8%); the contribution is reaching this on **unsupervised** $P(X)$.

Trade-off

CDT-MSW is more accurate per detected event because it has access to ground truth labels. SE-CDT

Backup: “Why not deep learning?”

Question: “Couldn’t a deep learning model solve this more accurately?”

Deep models for drift detection — limitations

- Need a large labelled dataset to train — the very thing we don’t have on a streaming IIoT pipeline.
- Higher inference latency per window than a kernel computation.
- No closed-form theoretical bound on the test statistic.
- Decision is hard to explain to a domain expert.

Why MMD / ShapeDD fits this problem

- Fully unsupervised — no training run before deployment.
- Theoretical guarantees: Triangle Shape Theorem, Type-I error from H_0 calibration.
- Decision rule is human-readable (a small tree of geometric features).
- One mechanism works across $P(X)$ shifts of arbitrary type.

Future direction: keep the kernel framework but plug in a learned feature extractor for high-dimensional inputs (Chapter 6).



Backup: Permutation test vs Gamma-null p-value

Question: “Removing the permutation test — is the Gamma-null p-value still trustworthy?”

Permutation test (original ShapeDD)

- Reshuffles indices many times per window.
- Empirical null, no distributional assumption.
- Each shuffle re-builds the kernel matrix $\rightarrow$ heavy compute.

Gamma-null p-value (IDW-MMD)

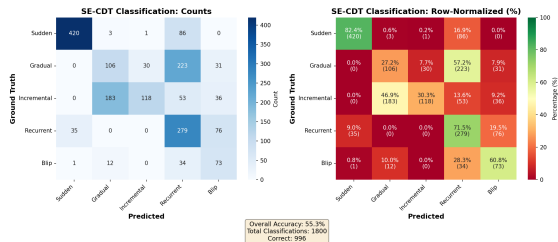
- Moment-matched Gamma fit to the null MMD distribution (Gretton et al. 2009).
- Estimates the first two moments from a small number of index shuffles, reusing the kernel matrix.
- Complexity $O(n^2)$ for the kernel plus a few cheap shuffles.

Verification: The detection F_1 remains comparable to the permutation baseline; the H_0 -calibration on three stationary distributions reports Standard MMD and IDW-MMD inside the 95% CI of $\alpha = 0.05$ (0.025–0.075), while the SE-CDT composite is anti-conservative at 0.080–0.140 (up to $\approx 2.8 \times \alpha$), reflecting peak-selection and sliding-window dependence rather than the per-candidate test itself. End-to-end runtime is slightly faster than the ShapeDD baseline.



Backup: “Subcategory accuracy 55.3% — is it close to random?”

Question: “With 5 classes, is $\sim 60\%$ close to random guessing?”



It is well above random:

- Random baseline across 5 classes is 20%.
- Micro SUB = 55.3% is $\sim 2.8\times$ random and well above the 95% CI of a random predictor ($\sim 23\%$).
- Performance is **very uneven across classes**:

Drift Type	Accuracy
------------	----------

Sudden	82.4%
--------	-------

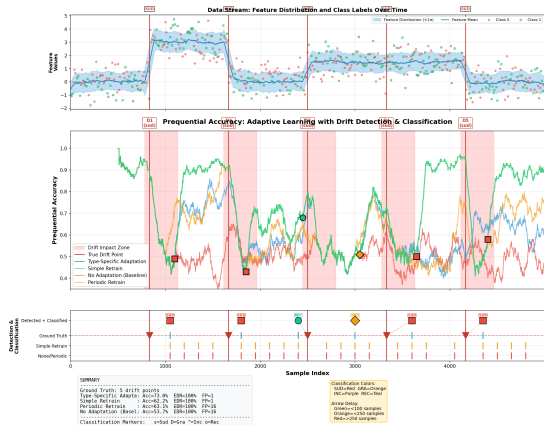
Incremental	30.3%
-------------	-------

Recurrent	71.5%
-----------	-------



Backup: “Is the adaptation gap statistically significant?”

Question: “Is Type-Specific really better, or is it run-to-run noise?”



Sudden ($N = 30$):

- Type-Specific: $70.78\% \pm 2.85$

Friedman test (paired by seed):

- Stepping: $\chi^2 = 78.12$, $p = 7.8 \times 10^{-17}$
- Sudden: $\chi^2 = 86.76$, $p = 1.1 \times 10^{-18}$
- Mixed: $\chi^2 = 76.32$, $p = 1.9 \times 10^{-16}$

$p \ll 0.05$ on all three scenarios; the ranking among 4 strategies is not random.

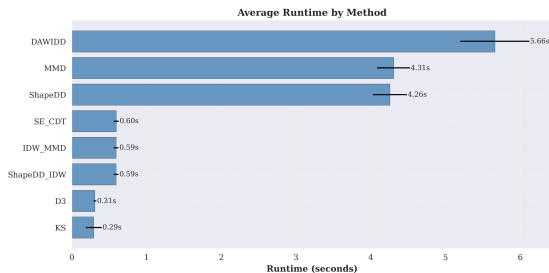
Remaining limits

- Only 3 scenarios (Stepping, Sudden, Mixed).
- No Gradual/Incremental adaptation scenario yet.
- No real-world dataset with verified drift labels.

Listed in **Chapter 6: Conclusion and Future Directions**

Backup: Computational Overhead & Runtime Analysis

Question: “What is the computational overhead of IDW-MMD compared to standard methods?”



Complexity & Efficiency:

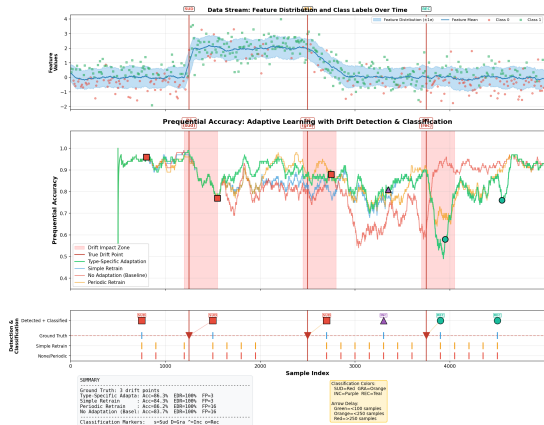
- **Standard MMD / IDW-MMD:** $O(n^2)$ matrix computation, but IDW computes weights once per window step.
- **Gamma-null Approximation** bypasses the permutation test entirely, running in constant extra shuffles.
- **ShapeDD-IDW** is roughly **7×** faster than the original permutation-based ShapeDD (0.59s vs 4.26s per 10k samples).

Streaming Suitability

In online deployments, the detector easily processes streams at over **16k samples/sec**, making its latency negligible in streaming Kafka pipelines.

Backup: Model Adaptation on Mixed Drift Scenario

Question: “How does the type-aware adaptation perform on complex, multi-drift streams?”



Key Insights for Mixed Streams:

- Mixed stream contains a sequence of Sudden, Gradual, and Incremental drifts.
- **Type-Specific (blue)** achieves the highest final accuracy by correctly routing Sudden to full-reset and Gradual to warm-start.
- **Simple Retrain (green)** suffers on Gradual periods by wiping past useful knowledge too early.
- **No Adaptation (red)** permanently degrades as distribution shifts compound.

Statistical Significance

Friedman test confirms the ranking is highly significant on the Mixed scenario ($\chi^2 = 76.32$, $p = 1.9 \times 10^{-16}$).

Backup: Why are Gradual (27.2%) and Incremental (30.3%) low?

Question: “The two slow types are the weak point — what exactly goes wrong?”

Two systematic confusions:

- 1 **Gradual** $\leftrightarrow$ **Incremental overlap**: Both land in ambiguous Variance Ratio bands ($\sim 47\%$ of Incremental read as Gradual).
- 2 **Saturation** $\rightarrow$ **Recurrent**: In repeated slow drifts, $P(X)$ stops changing. The post-drift window matches stored snapshots, labelling it Recurrent ($\sim 57\%$ of Gradual events).

Why CAT is unaffected

Both confusions stay *inside* PCD, preserving TCD/PCD accuracy (CAT = 80.1%). Confusing Gradual with Incremental is low-cost for adaptation.

Fix (future work)

Replace fixed VR thresholds with learned classifiers and separate Concept-Memory match step from shape classification.



Backup: “Is the adaptation gain Sudden-biased?”

Question: “Type-Specific wins +17pp — but the classifier is only $\sim 55\%$ on subtypes. Does the gain depend on the easy scenarios?”

Honest answer — yes, partly:

- The three adaptation scenarios (Stepping, Sudden, Mixed) are all **Sudden-dominated**, exactly where the classifier is strongest (Sudden 82.4%).
- So routing is correct *most of the time in this region*, and any reset on a true sudden drift helps regardless.
- We have **not** tested adaptation on Gradual/Incremental-heavy streams, where subtype accuracy drops to 27–30%.

Why the gain still holds

The category decision (TCD vs PCD, 80.1%) drives the adaptation policy, not the exact subtype. TCD→reset and PCD→fine-tune is the right call even when the subtype within PCD is wrong — the operational cost of confusing Gradual with Incremental is small (same update family).

Scope claim

We claim the adaptation benefit *for the sudden/recurrent regime*, not universally. Gradual/Incremental-heavy adaptation is listed as future work (Ch. 6).

